Personal Wardrobe Stylist

Introduction

As the world of technology advances at an unprecedented pace, artificial intelligence (AI) stands at the forefront, reshaping various industries. Its rapid evolution has unlocked transformative possibilities, and the fashion retail websites are poised to leverage this burgeoning potential in revolutionizing the fashion retail landscape.

Our team is embarking on a groundbreaking venture aimed at redefining the fashion retail experience through the fusion of cutting-edge AI technologies. Our mission is to create the Personal Wardrobe Stylist Application, an innovative solution that harnesses the power of AI to elevate how customers explore and discover fashion choices for every occasion.

The growth and advancements in AI have witnessed an incredible surge in recent years. Machine learning models, such as GPT (Generative Pre-trained Transformer) and CLIP (Contrastive Language-Image Pre-training), have showcased unparalleled capabilities in understanding and interpreting human queries, enabling a deeper and more intuitive interaction between technology and users.

In this era of rapid technological progression, we recognize the transformative potential AI holds in reshaping customer experiences. The Personal Wardrobe Stylist Application embodies our commitment to harnessing these cutting-edge AI advancements to provide customers with a seamless, intuitive, and personalized fashion exploration journey.

Through the integration of AI-driven technologies, the fashion retail market endeavors to empower customers with a unique and tailored shopping experience, delivering personalized fashion recommendations that resonate with individual preferences and event requirements.

1.1 Problem Statement

In today's dynamic world, navigating the sea of fashion options to find the perfect outfit for specific events can be overwhelming for customers. The challenge lies in seamlessly blending personal style preferences with event requirements while ensuring convenience and accessibility. We recognize this challenge and endeavors to streamline this process through an innovative and intelligent solution.

1.2 Solution

The proposed Personal Wardrobe Stylist Application integrates growing AI tools like OpenAI's GPT and CLIP to provide users with personalized fashion advice. Users can conveniently input details on the Streamlit app, enabling the AI to create custom fashion suggestions. The application will search through the user's uploaded wardrobe and the fashion retail website’s online store using AI-generated tags and descriptions stored in a database. Additionally, it will offer direct purchase links for the recommended clothing items.

1.3 Project Mission

Our mission is to empower the customers with an intelligent and convenient fashion solution. By amalgamating cutting-edge AI technologies and the extensive product database, we strive to deliver a personalized shopping experience that simplifies outfit selection and fosters greater customer satisfaction and loyalty.

This application will not only redefine the way customers shop for events but also position us as a pioneering force in integrating AI-driven solutions within the retail industry, setting new standards for personalized shopping experiences.

Purpose and Objectives

2.1 Purpose

Our project's fundamental aim is to provide the customers with immersive, personalized fashion recommendations by seamlessly fusing AI technology with the extensive product inventory. The application endeavors to simplify the often-daunting task of selecting the ideal outfit for diverse events, enriching customer satisfaction and engagement.

2.2 Objectives

1. Develop an intuitive, user-friendly interface enabling customers to effortlessly input event details and receive tailored fashion recommendations.

2. Implement and seamlessly integrate GPT to interpret user queries and generate personalized clothing suggestions.

3. Utilize CLIP technology for efficient product image retrieval, optimizing matches from the user's uploaded wardrobe.

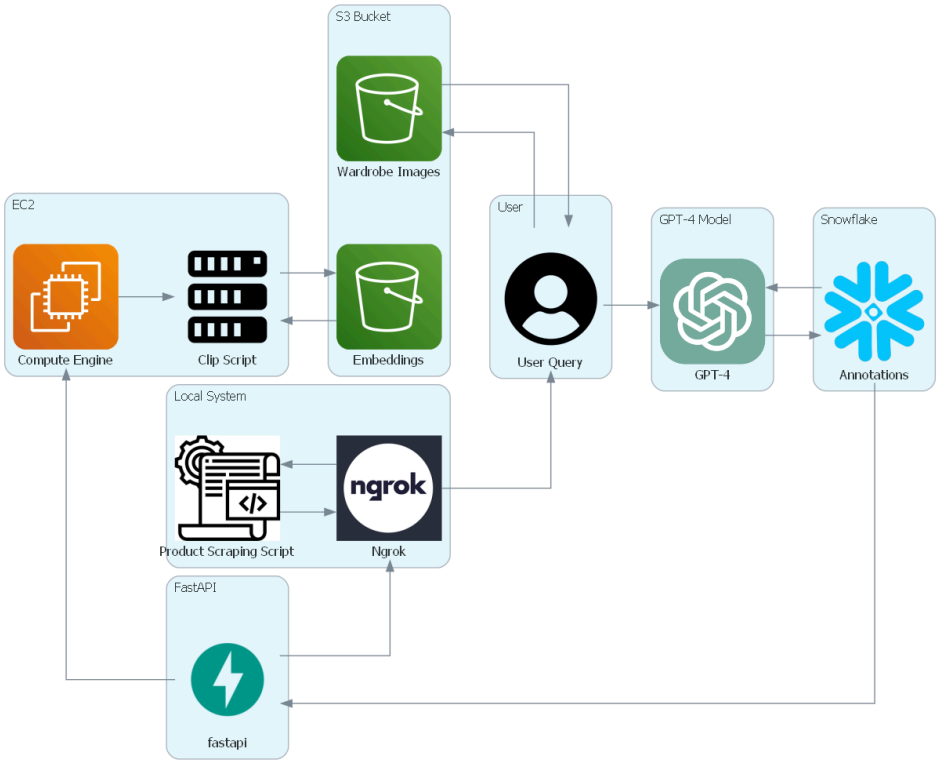
4. Design and implement a robust backend system facilitating seamless communication between AI models and product databases.

5. Offer customers refined product suggestions from fashion retail online stores based on AI-generated recommendations.

6. Ensure application optimization for scalability, high performance, and exceptional user experience.

Technical Overview and Architecture

3.1 Architecture Diagram



Personal Wardrobe Stylist Architecture

3.2 Overview

The project focuses on delivering personalized wardrobe recommendations to users through an application interface.The system utilizes OpenAI's GPT-4 model by sending user prompts alongside predefined prompt scripts. This approach ensures that the system comprehensively understands the scenarios, allowing GPT-4 to generate precise recommendations based on the provided context.This information is utilized to provide annotations for suggested clothing combinations.

Upon receiving these annotations, the system bifurcates into two pipelines:

Link Generator Script & Scraping Pipeline: Annotated suggestions are directed to a link generator script. This script initiates a scraping process, extracting recommended product links from the Macy's online store, based on the annotations received through Fast API. These links serve as recommendations for purchasing clothes online.
FastAPI Script on Amazon EC2 Instance & Image Pipeline: An EC2 instance hosts a FastAPI script. Here, utilizing OpenAI's CLIP (Contrastive Language-Image Pretraining) model, images related to recommended clothing combinations are fetched from an Amazon S3 database. These images are sourced from user-uploaded images stored previously.
Wardrobe: User-uploaded images are processed through GPT Vision, generating tags. These metadata details are then stored in Snowflake, while the images themselves are stored in Amazon S3. When the FastAPI script on the Amazon EC2 instance requests images related to clothing recommendations, Snowflake is queried based on the generated metadata. The retrieved metadata guides the retrieval of relevant images stored in Amazon S3, using CLIP to fetch and present images matching the recommended clothing combinations.

3.3 Workflow Summary

User Input: Users provide details through the application, triggering GPT-4 to understand and create recommendations

Annotation Generation: GPT-4 produces annotations for clothing combinations.

Link Generation & Scraping: One pipeline generates online product links based on annotations.

Image Retrieval & Recommendation: The other pipeline fetches images related to recommendations from user-uploaded data stored in Amazon S3.
User Output: Users receive personalized clothing recommendations from their wardrobe and online product links, facilitating convenient online shopping.

Technical Implementation Breakdown

4.1 Predefined Prompt

The predefined prompt, when combined with the user's input, is utilized to engage the GPT-4 model. This interaction serves as a crucial step in the process, directing the model's understanding towards generating tailored recommendations based on the specific scenario provided.

The following code snippet defines a template for image tagging tasks. It sets the context for an image tagger, emphasizing their professional expertise and the precision expected in completing the given task. The template outlines specific tags related to clothing attributes and asks for accurate and precise completion of these tags in a JSON format.

```
tagger = (
    "YOUR ROLE :\\n"
    " You are a professional Image tagger with significant experience in
this field."
    "This experience has made you a pro in this domain and there is no
chance you could do any sorts of inaccuracies."
    "TASK/TAGGING: \\n"
    "complete the tags for the given image from the template."
    "Do not return any text except json file. Be accurate and precise"

    "\n{"
    "\n  \"annotations\": ["
    "\n    {\"tag\": \"Clothing Type\", \"value\": \"\"},"
    "\n    {\"tag\": \"Color\", \"value\": \"\"},"
    "\n    {\"tag\": \"Pattern\", \"value\": \"\"},"
    "\n    {\"tag\": \"Season\", \"value\": \"\"},"
    "\n    {\"tag\": \"Length\", \"value\": \"\"},"
    "\n    {\"tag\": \"Occasion\", \"value\": \"\"},"
    "\n    {\"tag\": \"Fit\", \"value\": \"\"},"
    "\n    {\"tag\": \"Length\", \"value\": \"\"},"
    "\n    {\"tag\": \"Formal\", \"value\":\"Yes or No\"}"
    "\n  ]"
    "\n}\\n"
)
```

4.2 Scraping

This below code essentially scrapes Macy's website based on provided keywords, extracts image URLs and product URLs, and displays them if the request is successful.

Importing Libraries:

This section imports necessary libraries required for the script to function properly. The `requests` library is used to send HTTP requests, `BeautifulSoup` from `bs4` is employed for parsing HTML content, and `time` module is imported for managing time-related functionality.

```
import requests
from bs4 import BeautifulSoup
import time
```

Function to Fetch Product Information:

This section defines the `fetch_product_info` function, responsible for fetching product information from Macy's website based on provided keywords.

```
def fetch_product_info(*keywords):
    # Construct the base URL with provided keywords
    base_url = "https://www.macys.com/shop/search?keyword=" +
"+".join(keywords)

    # Set headers for the HTTP request
    headers = {
        'authority': 'www.macys.com',
        'accept':
'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apn
g,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9',
        'accept-language': 'en-GB,en-US;q=0.9,en;q=0.8',
        'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/118.0.0.0 Safari/537.36'
    }

    # Send a request to get the product information
    response = requests.get(base_url, headers=headers)

    if response.status_code == 200:
        soup = BeautifulSoup(response.text, 'html.parser')

        # Extract product containers
        product_containers = soup.find_all('div',
class_='productThumbnail')

        # Create a list to store product information
        products = []

        for container in product_containers:
            # Extract image URL
            img_element = container.find('img', class_='thumbnailImage')
            image_url = img_element['src'] if img_element and 'src' in
img_element.attrs else None

            # Extract product URL
            product_url_element = container.find('a',
class_='productDescLink')
```

```
            product_url = product_url_element['href'] if
product_url_element else None

            if image_url and product_url:
                products.append({
                    'image_url': image_url,
                    'product_url': product_url
                })

        time.sleep(2)

        return products

    else:
        print(f"Failed to fetch data. Status code: {response.status_code}")
        return None
```

4.3 Fast API

This code establishes an API using FastAPI for conducting image searches using the CLIP model and AWS S3 storage. It includes functionalities to load features and IDs from S3, encode search queries, find best image matches, and expose an endpoint for conducting image searches via HTTP POST requests.

Importing Libraries:

```
import streamlit as st
from PIL import Image
import torch
import clip
import numpy as np
import pandas as pd
from io import BytesIO
import boto3
from snowflake_list import annotations_list
from fastapi import FastAPI, HTTPException
from fastapi.responses import StreamingResponse
from typing import List
from pydantic import BaseModel
import uvicorn
import io
```

```
from fastapi.responses import JSONResponse
import requests
from bs4 import BeautifulSoup
import time
```

- The code imports various libraries and frameworks necessary for different functionalities within the application.
- Libraries such as Streamlit, PIL (Pillow), PyTorch, CLIP, FastAPI, boto3 for AWS, pandas, and others are imported to handle tasks related to data processing, APIs, image manipulation, web scraping, and more.

Initializing FastAPI:

```
app = FastAPI()
```

The FastAPI instance (`app`) is created, which will handle endpoints for API interactions.

Defining Data Models:

```
class ProductInfo(BaseModel):
    image_url: str
    product_url: str
```

`ProductInfo` is a data model used for API request/response validation. It defines the structure of data (image URL and product URL) expected for certain API requests.

Load CLIP Model and AWS Credentials:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = clip.load("ViT-B/32", device=device)[0]

# Collect AWS secrets
bucket_name = st.secrets.aws_credentials.bucket_name
service_name = st.secrets.aws_credentials.service_name
aws_access_key_id = st.secrets.aws_credentials.aws_access_key_id
aws_secret_access_key = st.secrets.aws_credentials.aws_secret_access_key
region_name = st.secrets.aws_credentials.region_name

# S3 client setup
s3_client = boto3.client(
    service_name=service_name,
    aws_access_key_id=aws_access_key_id,
```

```
    aws_secret_access_key=aws_secret_access_key,
    region_name=region_name
)
```

It initializes the CLIP model for image-text embeddings and sets up an AWS S3 client using credentials retrieved from Streamlit's secrets.

Loading Image Features and IDs from S3:

```
def load_features_ids():
    # ...
    return image_features, image_ids
```

This function retrieves image features and IDs stored in AWS S3.

Fetching Product Information from a Website:

```
def fetch_product_info(base_url):
    # ...
    return products
```

The `fetch_product_info` function sends a request to a given URL, scrapes product information (image URL and product URL) from the website, and returns the collected data.

Encoding Search Query and Finding Best Matches:

```
def encode_search_query(search_query):
    # ...
    return text_encoded

def find_best_matches(text_features, image_features, image_ids,
results_count=2):
    # ...
    return [image_ids[i] for i in best_image_idx[:results_count]]

def search(search_query, results_count=2):
    # ...
    return find_best_matches(text_features, image_features, image_ids,
results_count)
```

These functions involve encoding search queries, calculating similarities between text and image features, and finding the best matching images based on search queries.

FastAPI Endpoints:

```
@app.post("/image-search")
def image_search(query: dict):
    # ...
    return {"image_ids": results}

@app.post("/get_product_info")
async def get_product_info(base_url: str):
    # ...
    return {"products": result}
```

- These are the FastAPI endpoints (`/image-search` and `/get_product_info`) that handle POST requests.
- `image_search` endpoint receives a query and returns image IDs based on the search.
- `get_product_info` endpoint accepts a base URL, fetches product information from a website, and returns the collected products' details.

Running the FastAPI Server:

```
if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

This block of code starts the FastAPI server using Uvicorn on host `0.0.0.0` and port `8000` when the script is directly executed.

4.4 Image annotations

This code processes images, encodes them, combines with text content, sends requests to the GPT-4 Vision API, and saves the API responses paired with image names into a JSON file. It also includes handling API rate limitations by introducing a delay between requests.

Imports and Setup:

```
import os
import openai
import base64
import requests
```

```
import json
import time
from keys import keys
from prompt import tagger
```

This section imports required libraries (`os`, `openai`, `base64`, `requests`, `json`, `time`) and imports `keys` and `tagger` from separate files (`keys.py` and `prompt.py`).

Function to Encode a Single Image:

```
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode('utf-8')
```

This function encodes an image to base64 format using its file path.

OpenAI Configuration and File Handling:

```
openai.api_key = keys()
filename = 'responses.json'
```

Sets up the OpenAI API key and specifies the filename where the response content will be stored.

Function to Save Response Content:

```
def save_response_content(image_name, content):
    # Saves image name and response content to a JSON file
    # Handles both creating the file and updating its contents
```

This function saves response content along with the corresponding image name to a JSON file.

Image Processing and API Request Handling:

```
folder_path = "Wardrobe"

for image_filename in os.listdir(folder_path):
    if image_filename.endswith(('.jpg', '.jpeg', '.png')):

        # Encoding image to base64
```

```
        image_path = os.path.join(folder_path, image_filename)
        base64_image = encode_image(image_path)

        # Prepare image and text content for API payload
        image_content = {"type": "image_url", "image_url": {"url":
f"{base64_image}"}}
        text_content = {"type": "text", "text": tagger}
        combined_contents = [text_content] + [image_content]

        # Construct API request payload
        payload = {"model": "gpt-4-vision-preview", "messages": [{"role":
"user", "content": combined_contents}], "max_tokens": 4000}

        # Make POST request to OpenAI GPT-4 Vision API
        response =
requests.post("https://api.openai.com/v1/chat/completions",
headers={"Content-Type": "application/json", "Authorization": f"Bearer
{openai.api_key}"}, json=payload)

        # Handling API response
        if response.status_code == 200:
            # Extract and save response content
            save_response_content(image_filename,
response.json()['choices'][0]['message']['content'])
        else:
            # Print error message if API request fails
            print(f"Failed to get a successful response for
{image_filename}. Status code: {response.status_code}")

        # Adding sleep duration to handle API rate limitations
        time.sleep(180)
```

-This section iterates through images in the "Wardrobe" folder.

- Encodes each image to base64 and prepares image and text content.

- Constructs an API request payload combining text and image content.

- Sends a POST request to OpenAI's GPT-4 Vision API for completion and processes the response.

- Handles the API rate limitations by introducing a delay between API calls.

4.5 Json to dataframe converter

This script essentially reads JSON data, extracts image annotations, converts it to a DataFrame, and finally saves it as a CSV file for further analysis or usage.

Imports and Setup:

```
import pandas as pd
import json
```

Imports necessary libraries (`pandas`, `json`) for data manipulation and JSON handling.

Function to Convert JSON to DataFrame:

```
def json_to_df_converter(json_data):
    # Extracting image and annotations from JSON
    data = []
    for entry in json_data:
        image = entry['image']

        # Extracting JSON content within triple backticks
        content_start = entry['response_content'].find("```json") + 7
        content_end = entry['response_content'].rfind("```")

        if content_start != -1 and content_end != -1:
            json_content =
entry['response_content'][content_start:content_end].strip()

            try:
                # Removing extra characters and decoding JSON
                response_content = json.loads(json_content.replace("\n",
"").replace("\r", "").replace("```", ""))
                annotations = response_content['annotations']
                data.append({'image': image, 'annotations': annotations})
            except json.decoder.JSONDecodeError as e:
                print(f"Error decoding JSON for image {image}: {e}")
        else:
            print(f"Could not find valid JSON content for image {image}")

    # Creating DataFrame
    df = pd.DataFrame(data)
    return df
```

This function processes JSON data to extract image names and annotations. It iterates through the JSON entries, locates the JSON content within triple backticks, and attempts to extract annotations for each image. It then constructs a DataFrame containing image names and their respective annotations.

Main Execution:

```
if __name__ == '__main__':
    # Load JSON data from file
    with open('responses.json', 'r') as file:
        json_data = json.load(file)

    # Convert JSON to DataFrame using the defined function
    df = json_to_df_converter(json_data)

    # Save DataFrame as CSV file
    df.to_csv("tags.csv", index=False)
```

The script reads the JSON data from the file, calls the `json_to_df_converter()` function to convert JSON to a DataFrame, and then saves the DataFrame as a CSV file named `tags.csv`.

4.6 Snowflake Integration

This code connects to a Snowflake database, fetches annotations from the `wardrobe_tags` table, converts them into a list, and closes the database connection afterward. It demonstrates how to utilize SQLAlchemy to interact with a Snowflake database in Python.

Importing Libraries:

```
import pandas as pd
from sqlalchemy import create_engine
```

Imports the necessary libraries (`pandas` and `create_engine` from `sqlalchemy`) for data manipulation and establishing a connection to the database.

Snowflake Database URL:

```
snowflake_url =
"snowflake:/xxxxxx:xxxxxx@PXTNTMC-FTB58373/CLOTHING_TAGS/PUBLIC?warehouse=C
OMPUTE_WH&role=ACCOUNTADMIN"
```

Defines the URL to connect to the Snowflake database. It includes credentials (`username:password`), account name, database name, warehouse, and role.

Function to Retrieve Annotations List:

```
def annotations_list(snowflake_url):
    engine = create_engine(snowflake_url)

    try:
        with engine.connect() as conn:
            sql_query = "select annotations from wardrobe_tags;"
            result2 = conn.execute(sql_query)

            annotations_data = pd.DataFrame(result2.fetchall(),
columns=result2.keys())
            annotations_data = annotations_data['annotations'].tolist()

    except Exception as e:
        print(e)
    finally:
        conn.close()

    return annotations_data
```

- `annotations_list` function establishes a connection to the Snowflake database using the provided URL.
- Executes an SQL query (`"select annotations from wardrobe_tags;"`) to fetch data from the `wardrobe_tags` table.
- Retrieves the fetched data into a Pandas DataFrame (`annotations_data`) and converts it to a list (`annotations_data['annotations'].tolist()`).
- It handles potential exceptions and ensures to close the connection (`conn.close()`), regardless of success or failure.

Main Block:

```
if __name__ == "__main__":
    annotations_list(snowflake_url)
```

- This block checks if the script is directly executed (not imported as a module).
- Invokes the `annotations_list` function to retrieve the annotations from the Snowflake database when the script runs.

4.7 Fast API for recommendation links

This code integrates web scraping functionality to fetch product information from a given URL and creates a FastAPI endpoint (`/get_product_info`) to serve this information via an API using ngrok to make it accessible through a public URL.

Importing Libraries:

```
from pyngrok import ngrok
from fastapi.middleware.cors import CORSMiddleware
import nest_asyncio
import uvicorn
from bs4 import BeautifulSoup
import requests
import time
from fastapi import FastAPI, JSONResponse
```

Imports necessary libraries such as `pyngrok` for creating public URLs, `FastAPI` for building APIs, `nest_asyncio` for handling asyncio events, `uvicorn` for running the FastAPI server, and others for web scraping and handling API responses.

Function to Fetch Product Information:

```
def fetch_product_info(base_url):
    # ... (code to send a request to the given base_url and extract product
information)
    return products
```

- `fetch_product_info` function uses `requests` to fetch product information from a given `base_url` using web scraping techniques (using `BeautifulSoup`) to extract image URLs and product URLs.
- It then stores this information in a list of dictionaries (`products`) containing image URLs and corresponding product URLs.

FastAPI Setup:

```
app = FastAPI()

@app.post("/get_product_info")
```

```
async def get_product_info(base_url:str):
    result = fetch_product_info(base_url)
    if result:
       return {"products": result}
    return JSONResponse(content={"message": "Failed to fetch data"},
status_code=500)
```

- Initializes a FastAPI instance (`app`) and creates a single API endpoint (`/get_product_info`) that accepts POST requests.
- The endpoint receives a `base_url` parameter, calls the `fetch_product_info` function, and returns the extracted product information in JSON format if successful.

ngrok Configuration and Running the FastAPI Server:

```
ngrok_tunnel = ngrok.connect(8000)
print('Public URL:', ngrok_tunnel.public_url)
nest_asyncio.apply()
uvicorn.run(app, port=8000)
```

- Creates an ngrok tunnel to expose the local FastAPI server running on port 8000 to a public URL.
- Prints the public URL generated by ngrok.
- `nest_asyncio.apply()` is used to patch asyncio even though it's not typically required for running FastAPI.
- Starts the FastAPI server using `uvicorn` on port 8000.

4.8 Streamlit Main page

The provided code is a Streamlit application that serves as an interface for the "Personal Wardrobe Stylist" application.

Importing Libraries:

```
import streamlit as st
```

Imports the Streamlit library for building web applications with Python.

Defining the Main Function:

```
def main():
    # Title and tag line
```

```
    st.title("Personal Wardrobe Stylist")
    st.write("Let Your Event Speak Your Style!")

    # Overview text
    st.header("Overview")
    st.write("Embarking on a revolutionary journey in fashion
technology...")
```

- The `main()` function sets up the Streamlit application.
- It includes sections for the title, tagline, and an overview of the application's purpose, displayed using Streamlit's `title`, `write`, and `header` functions.

Displaying a Video:

```
 # Video
    st.header("Feature Highlight")
    st.video("xxxxxxx")
```

This section adds a header and embeds a video into the application using Streamlit's `video` function.

Providing Links to User and Developer Guides:

```
    # User Guide
    st.header("User Guide")
    st.write("Access the User Guide here:")
    user_guide_url =
"https://codelabs-preview.appspot.com/?file_id=1rK05tYKUVYfXsGXxTgk34mNUvEn
JMNN64T21XHXgxAM#0"
    st.markdown(f"[User Guide]({user_guide_url})")

    # Developer Guide
    st.header("Developer Guide")
    st.write("Access the Developer Guide here:")
    developer_guide_url =
"https://codelabs-preview.appspot.com/?file_id=1GLvJAY5Ad_vSvI_1TGS0YKVMwm8
oBR0tY2SlN_XIitk#0"
    st.markdown(f"[Developer Guide]({developer_guide_url})")
```

These sections create headers and display links to the User Guide and Developer Guide using Streamlit's `write` and `markdown` functions.

Running the Application:

```
if __name__ == "__main__":
    main()
```

This block checks if the script is directly executed (not imported as a module) and calls the `main()` function to run the Streamlit application.

4.9 Streamlit Stylist page

The code aims to create a Streamlit application that interacts with GPT-3.5 to generate fashion recommendations, fetch product information and images, but some implementation details are missing, requiring additional development or integration for a fully functional application.

Imports:

```
import streamlit as st
from openai import OpenAI
import json
from PIL import Image
from io import BytesIO
import boto3
from snowflake_list import annotations_list
from macys_items import fetch_product_info
import requests
```

These libraries enable Streamlit functionalities, interaction with OpenAI's API, handling JSON data, image processing, AWS interaction, and custom functions.

API URLs and AWS Secrets:

```
url_clip = "http://3.133.150.2/image-search"
url_product =
"https://97ea-2601-19b-d81-6a0-d165-7c21-9710-fc51.ngrok.io/get_product_inf
o"
```

These URLs are endpoints used for making API requests, and AWS secrets might be employed to access AWS services (not fully detailed in this snippet).

Function to Interact with GPT-3.5

```
def interact_with_gpt(question, key, role=role):
    # Function to interact with GPT-3.5 using OpenAI's API
    return answer
```

This function interacts with OpenAI's GPT-3.5 model to generate a response based on a given question, API key, and predefined role.

Function to Display Images from AWS S3

```
def display_images_from_s3(image_ids):
    # Function to fetch and display images from an AWS S3 bucket
    pass  # Placeholder for logic to display images from S3
```

This function aims to retrieve and display images stored in an AWS S3 bucket based on provided image IDs.

Streamlit User Interface and Input Handling

```
def main():
    # Streamlit UI code to get user input
    return user_input
```

This function generates the Streamlit user interface, likely including a sidebar or input box to capture user input.

User Input Processing and GPT Interaction

```
if __name__ == "__main__":
    user_input = main()
    response = interact_with_gpt(user_input, openai_key)
    # Further processing of GPT-3.5 response and display of recommendations
    # API calls to fetch images and product recommendations
```

This section retrieves user input from the Streamlit interface, interacts with GPT-3.5 using the provided input, and manages the AI-generated response for providing recommendations.

Exception Handling
Throughout the code, try-except blocks are used to manage potential errors that might occur during interaction with APIs, data retrieval, or other processes.

4.10 Streamlit Wardrobe page

This code creates a Streamlit app that interacts with AWS S3, enabling users to view wardrobe images and add new images to the wardrobe by uploading them. It uses Streamlit's UI components to build an interactive web application interfacing with AWS S3.

Importing Necessary Libraries:

```
import boto3
import streamlit as st
import botocore.exceptions
```

- `boto3` is the Amazon Web Services (AWS) SDK for Python.
- `streamlit` is used for building web applications with Python.
- `botocore.exceptions` is used to handle exceptions related to AWS services.

Accessing AWS Credentials from Streamlit Secrets:

```
bucket_name = st.secrets.aws_credentials.bucket_name
service_name = st.secrets.aws_credentials.service_name
aws_access_key_id = st.secrets.aws_credentials.aws_access_key_id
aws_secret_access_key = st.secrets.aws_credentials.aws_secret_access_key
region_name = st.secrets.aws_credentials.region_name

s3_client = boto3.client(
    service_name=service_name,
    aws_access_key_id=aws_access_key_id,
    aws_secret_access_key=aws_secret_access_key,
    region_name=region_name
)
```

This section retrieves AWS credentials (bucket name, service name, access key, secret key, region) from Streamlit Secrets and creates an S3 client using `boto3`.

Function to Display Wardrobe Images:

```
def display_wardrobe(all_image_ids):
    # Function to display images stored in AWS S3 bucket using Streamlit's
'image()' function.
```

`display_wardrobe()` is a function designed to display wardrobe images. It takes a list of image IDs as input and displays them using Streamlit's `image()` function.

Listing Images in a Specified Folder in AWS S3:

```
def list_images_in_folder(folder_name):
    # Function to list images in the specified folder in the AWS S3 bucket.
```

`list_images_in_folder()` is a function that lists images in a specified folder in the AWS S3 bucket. It retrieves image URLs from the folder.

Streamlit App Structure:

```
def main():
    # Main function to create the Streamlit app.
```

`main()` function sets up the Streamlit app's user interface and functionality.

Displaying Wardrobe Images in the Streamlit UI:

```
# Displaying wardrobe images based on user selection
if selected_option == "Show My Wardrobe":
    # Display the wardrobe images fetched from S3 in Streamlit columns.
else:
    # Allow users to add images to the wardrobe by uploading them.
```

Depending on the user's selection ("Show My Wardrobe" or "Add Image into Wardrobe"), the Streamlit app displays wardrobe images or provides an option to add new images.

Uploading Images to AWS S3:

```
# Uploading images to the wardrobe folder in AWS S3
if uploaded_file:
    if st.button("Add Image"):
        # Upload the selected image file to the specified folder in the AWS
S3 bucket.
```

When the user uploads an image and clicks "Add Image," the selected image file is uploaded to the specified folder in the AWS S3 bucket.

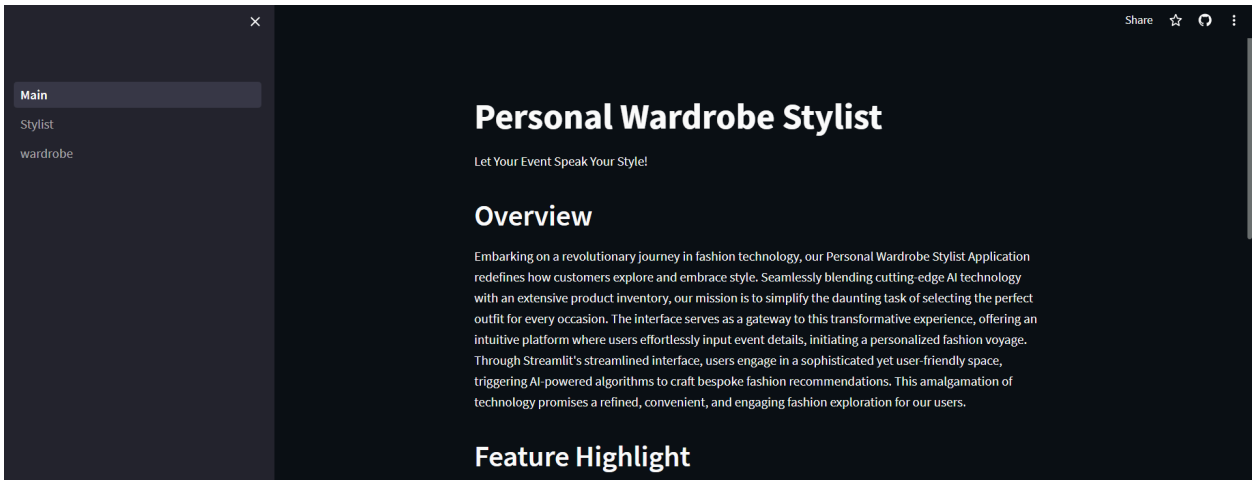
Exception Handling:

```
except botocore.exceptions.NoCredentialsError:
    st.error("AWS credentials not available.")
except Exception as e:
    st.error(f"Error adding image: {e}")
```

Exception handling to manage errors related to AWS credentials and S3 operations.

Application Execution Summary

- Upon accessing the application, you'll encounter the dashboard showcasing various options.
- Explore the menu tabs: "Home," "Stylist," "Wardrobe."



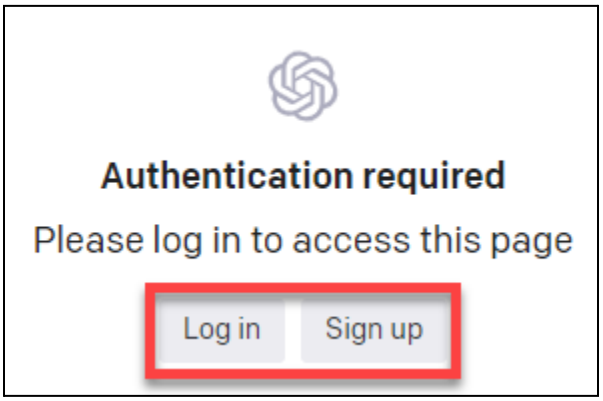
Using the Application

Upon accessing the Stylist Tab

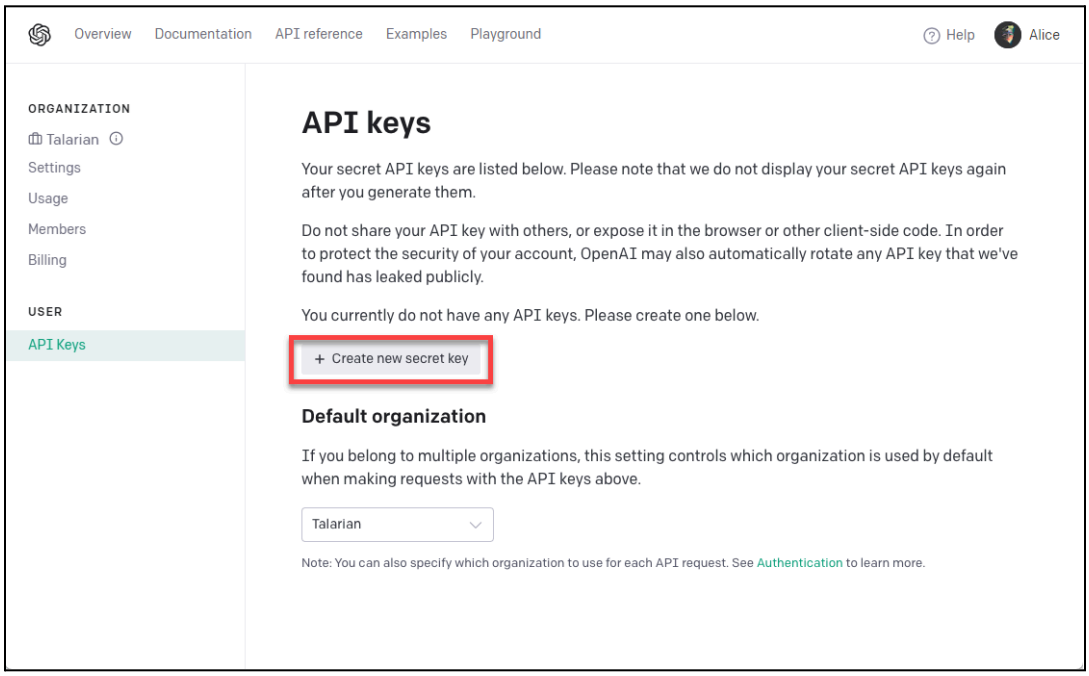
- Open AI Key

  Input your OpenAI access key. If you haven't obtained a key yet, please refer to the steps below to generate one.

  1. Log in or sign up to the OpenAI platform.

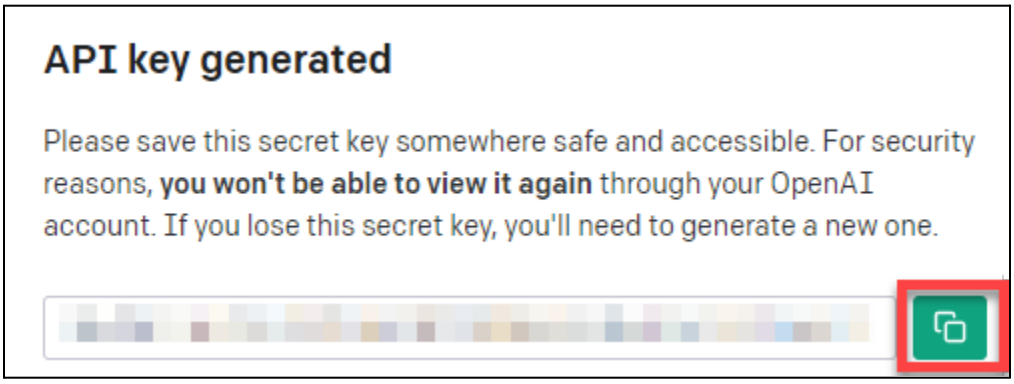


2. Click Create new secret key.



You can check that your key is working by running a prompt in the playground.If your free trial is expired, inactive, or not working, you must set up a paid account in the billing dashboard.

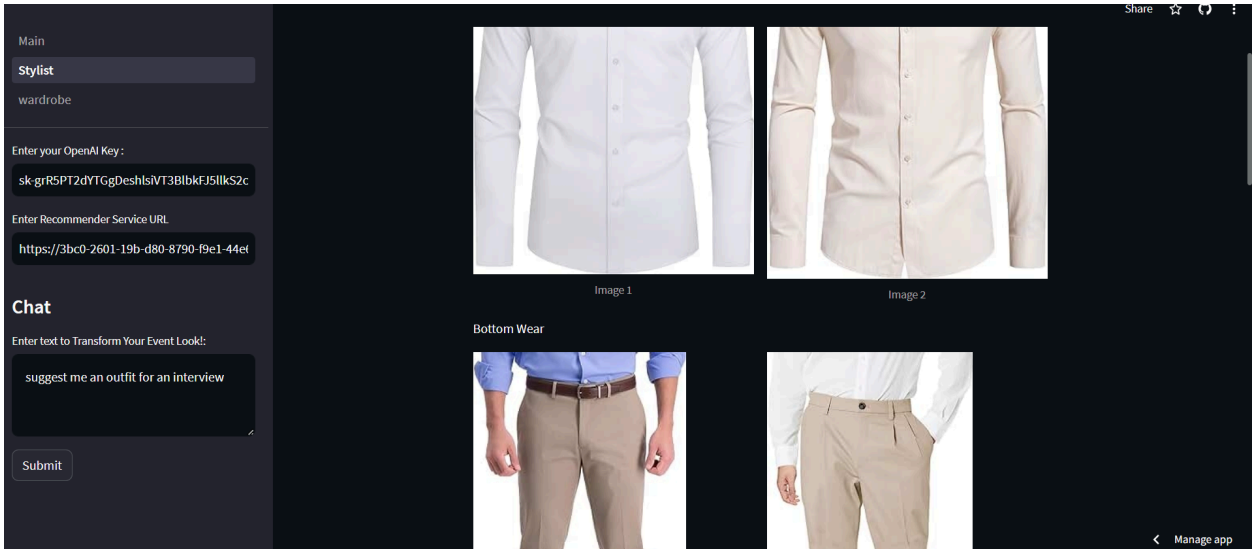
3. Copy your key to your clipboard.

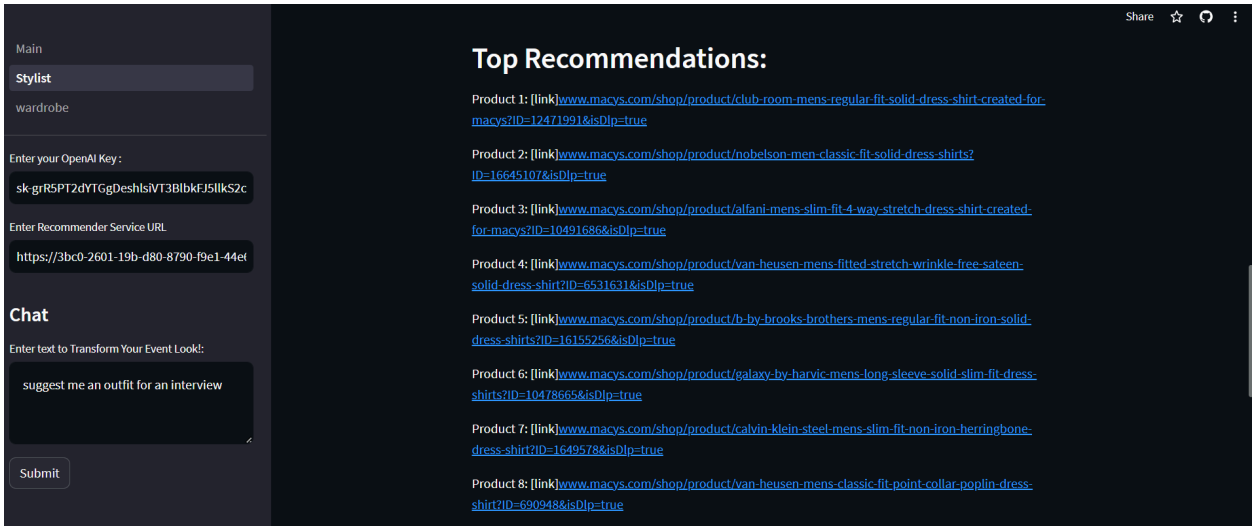


- Start the outfit recommendation search

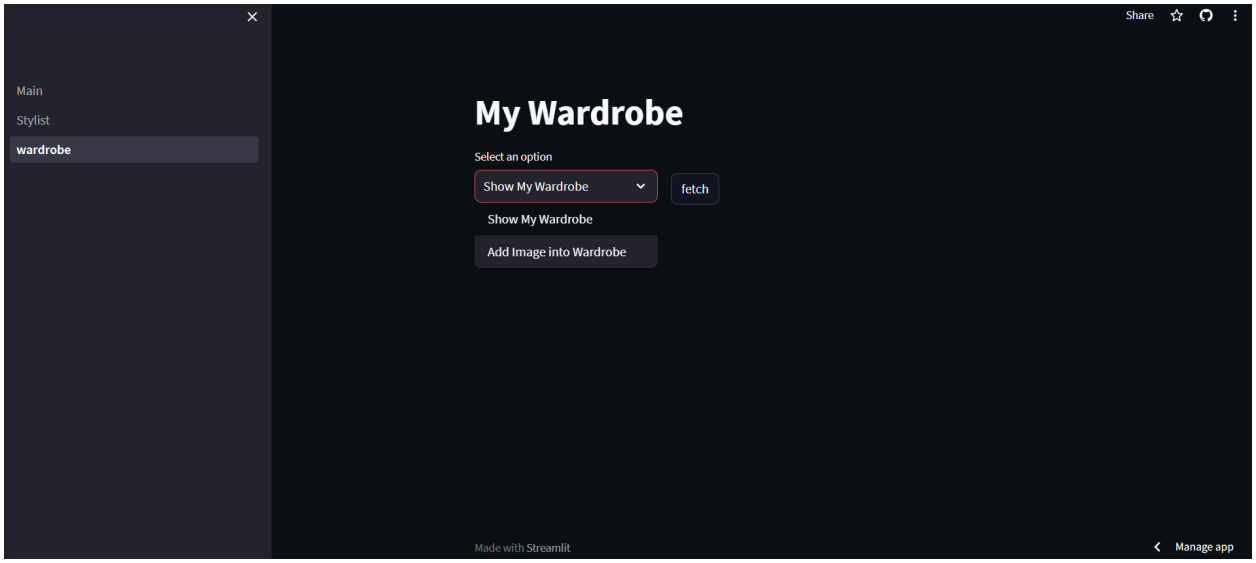
1. Input the text to receive personalized recommendations and proceed by selecting the 'Search' option.

2. In the output, you'll discover AI-generated recommendations for both top and bottom wear from your wardrobe, accompanied by quirky reasons suggested by the AI on why you should wear them. Additionally, you'll find recommendation links for online purchase options.



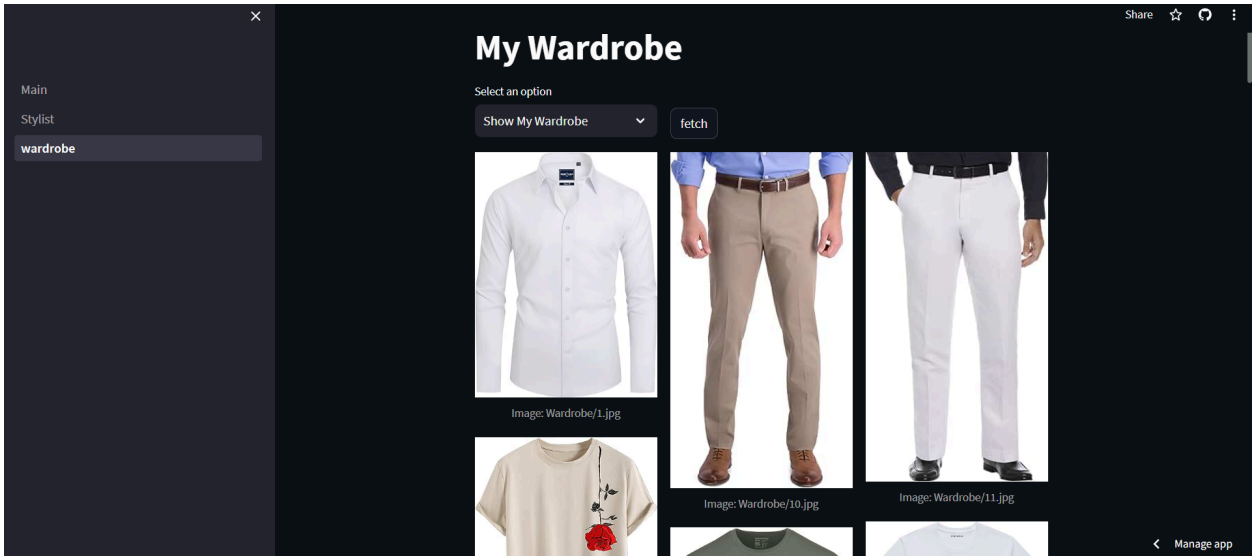


Upon accessing the Wardrobe Tab



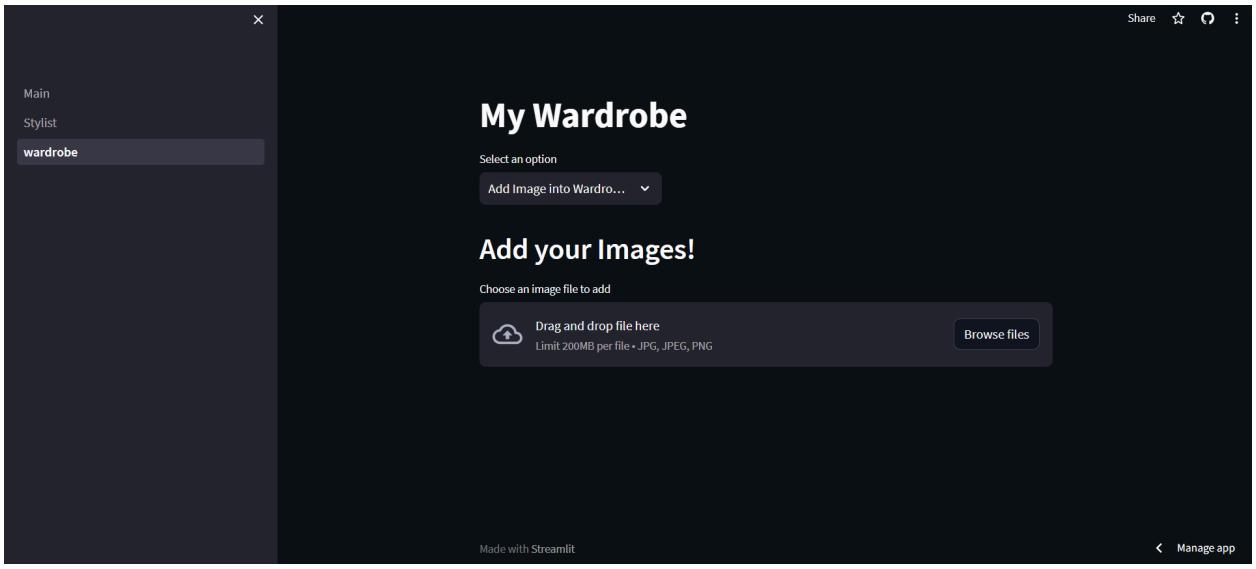
- Fetch Images

  You can access the images already uploaded to your wardrobe by selecting the 'Fetch Images' option.



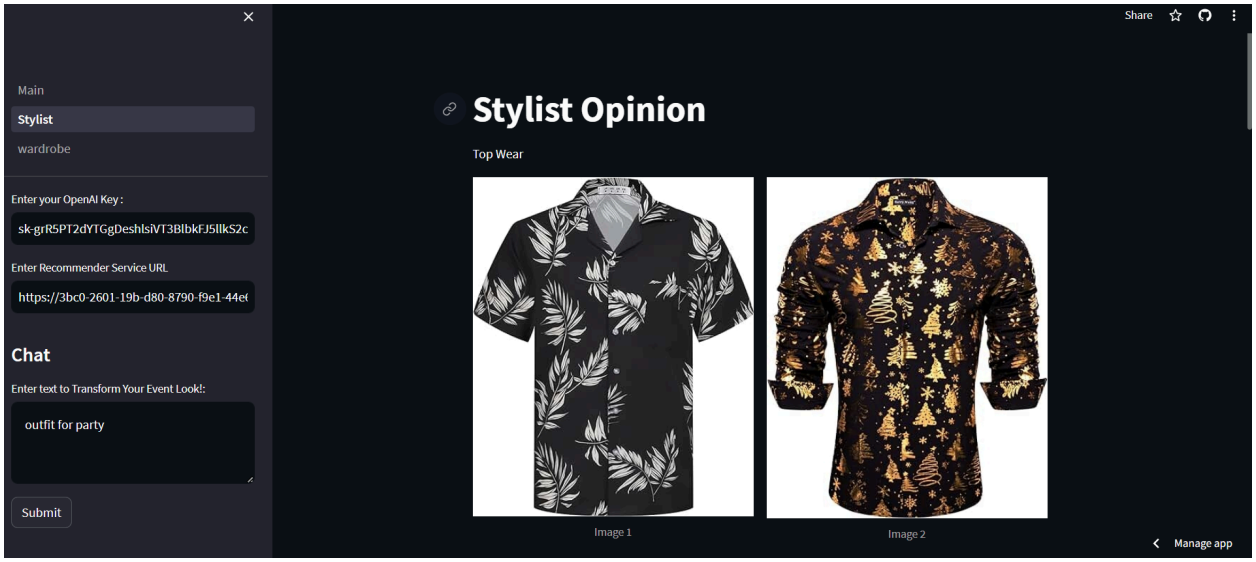
- Upload Images

  To add images from your device or computer, simply click on the 'Upload Image' option.

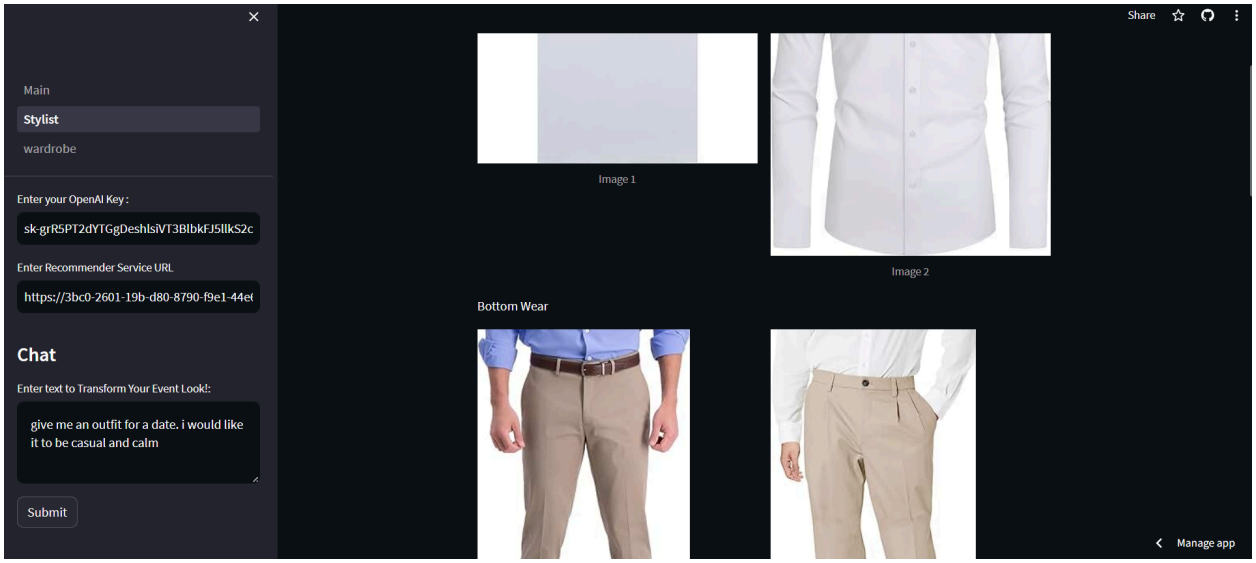


Test Cases

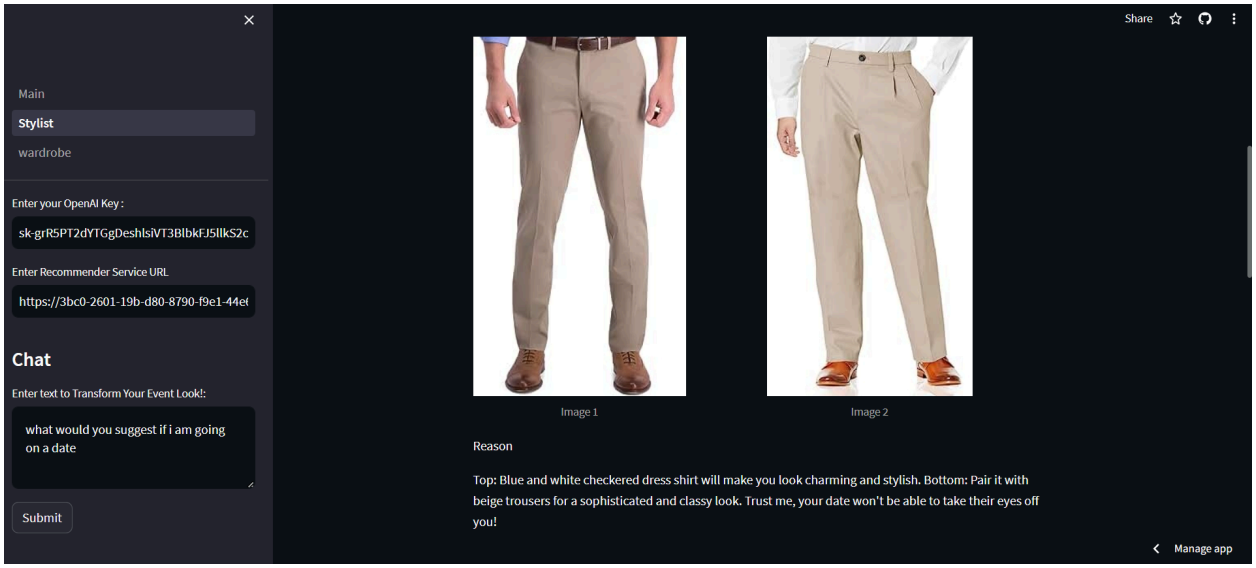
1. You can search by giving a simple query 'outfit for a party'



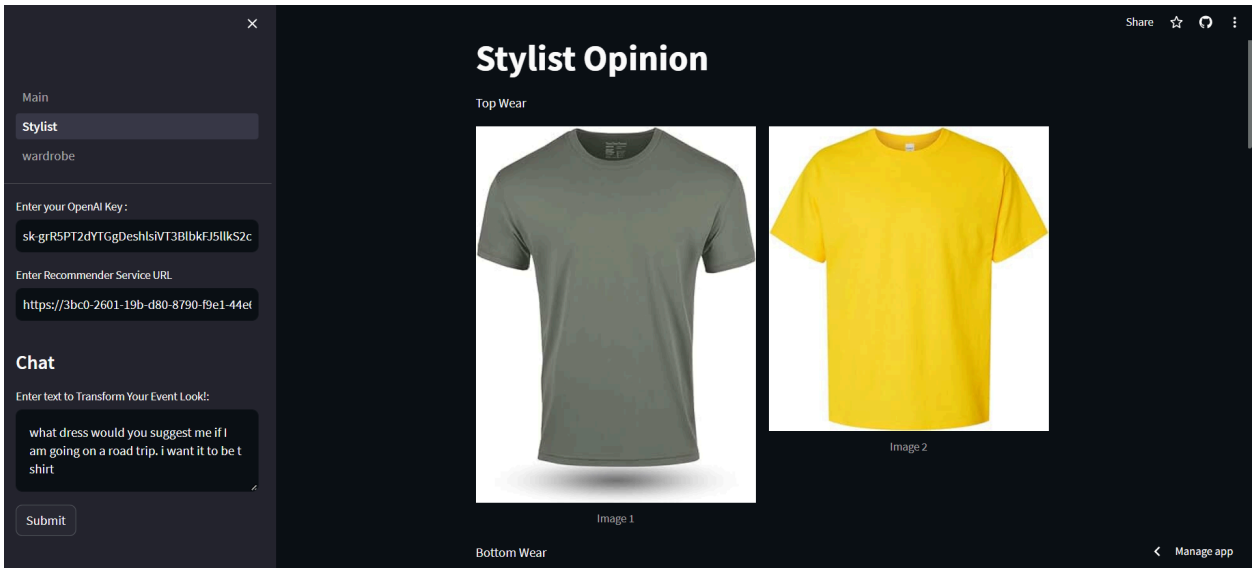
2. You can search by specifying your preferences, such as 'Give me an outfit for a date. I would like it to be casual and calm'



3. You can search by asking a question like, 'What would you suggest if I am going on a date?

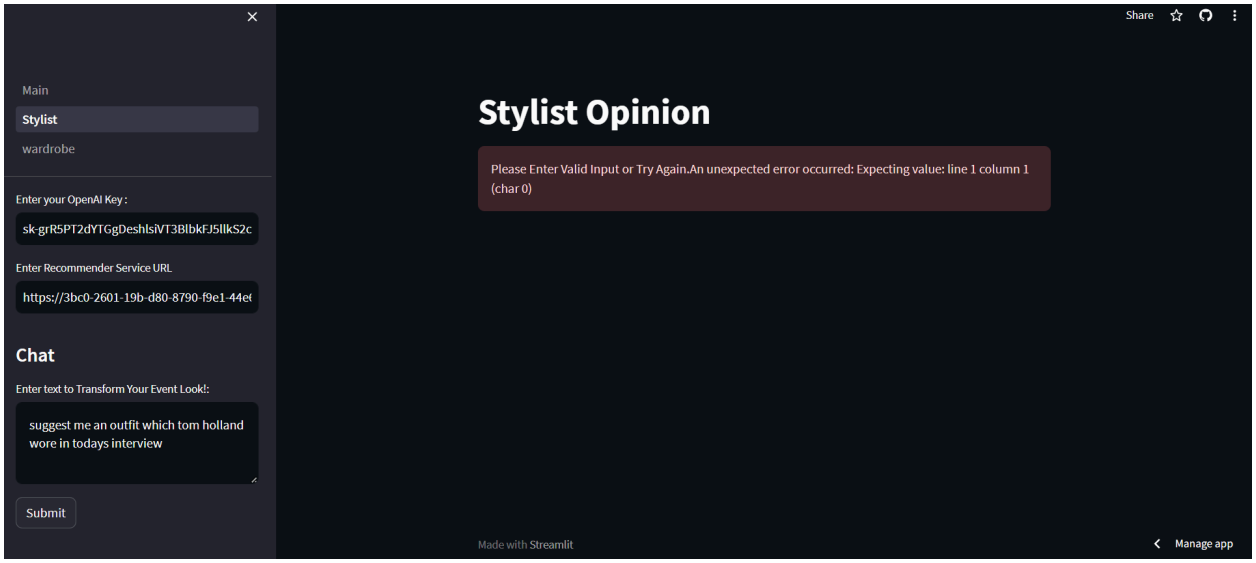


4. You can search by asking the question, "What dress would you suggest me if I am going on a road trip? I want it to be a T-shirt.



5. You cannot search by providing a single word like 'Birthday'

6. You cannot search by entering gibberish

7. You cannot conduct searches by providing mathematical conditions

8. You cannot search by providing statements such as 'suggest me an outfit that Tom Holland wore in today's interview'